SENTIMENT ANALYSIS WITH NLP & TRANSFORMERS

AI ACADEMIA — Artificial Intelligence & Machine Learning Internship Project Report (1-Month Track)

Intern Name: Mustina Himakiran	**Email ID:** mhimakiran123@gmail.com
Program Track: AI & Machine Learning (NLP & Transformers)	**Submission Date:** November 1, 2026

Primary Objective of the Project: To design, implement, and benchmark an end-to-end multi-class Sentiment Analysis system using Natural Language Processing (NLP) preprocessing pipelines, classical machine learning vectorization baselines (TF-IDF with Logistic Regression and Linear SVM), and fine-tuned pre-trained Transformer architectures (DistilBERT and RoBERTa) to accurately classify customer feedback into Positive, Negative, and Neutral sentiment categories.

1. Executive Summary & 1-Month Project Architecture

Automated sentiment analysis of unstructured customer reviews is a foundational capability in modern product analytics, brand reputation monitoring, and decision intelligence. In real-world e-commerce and digital service platforms, user feedback frequently blends domain-specific terminology with informal punctuation, HTML markup artifacts, negations (*"not bad at all"*), and multi-clause contrastive pivots (*"shipping was fast, but the device failed after two hours"*). Traditional lexical and bag-of-words models struggle to model word order and syntactic scope, leading to systematic misclassifications on nuanced or mixed-polarity reviews.

This 1-month internship project addresses these challenges by developing a unified, reproducible Natural Language Processing (NLP) and Deep Learning benchmark across a multi-class corpus of **1,800 annotated customer reviews** spanning three polarity categories: **Positive** (741 reviews, 41.2%), **Negative** (698 reviews, 38.8%), and **Neutral** (361 reviews, 20.1%). Executed across four structured weekly milestones aligned with the AI Academia curriculum, the project progresses from raw text exploration and negation-preserving normalization to sublinear TF-IDF classical baselines and culminates in fine-tuning bidirectional Transformer encoders (`distilbert-base-uncased` and `roberta-base`). On the stratified 20% holdout test split (N = 362), fine-tuned **DistilBERT reaches 94.20% accuracy (93.49% Macro-F1)** and **RoBERTa-base achieves 95.86% accuracy (95.18% Macro-F1)**, reducing overall classification error by more than 52% compared to the Logistic Regression TF-IDF baseline (87.85% accuracy).

Internship Timeline	Curriculum Module	Core Engineering Deliverables & Technical Milestones
Week 1 (Days 1–7)	Lessons A & B: Text Exploration & Cleaning	Corpus ingestion (N = 1,800), class balance profiling, regex stripping of HTML/URLs/user handles, and negation-preserving stopword normalization.

Internship Timeline	Curriculum Module	Core Engineering Deliverables & Technical Milestones
Week 2 (Days 8–14)	Lesson C: Classical NLP & Vectorization	Unigram/bigram sublinear TF-IDF feature extraction (3,500 features) and baseline benchmarking across Naive Bayes, Random Forest, Logistic Regression, and Linear SVM.
Week 3 (Days 15–21)	Lesson D: Transformer Fine-Tuning	Subword tokenization (WordPiece & Byte-Level BPE), sequence classification head initialization, and 4-epoch AdamW fine-tuning with linear warmup scheduling.
Week 4 (Days 22–30)	Lesson E: Evaluation & Error Analysis	Holdout confusion matrix diagnostics, contrastive clause attention analysis, latency vs. accuracy trade-off profiling, and production deployment packaging.

2. Week 1 (Lesson B): Corpus Exploration & Negation-Preserving Preprocessing

Raw user-generated text collected from web portals contains substantial non-semantic noise that inflates vocabulary cardinality without contributing discriminative sentiment signal. Exploratory profiling of our 1,800-review dataset revealed embedded HTML tags (`<br/>`, `<div>`), promotional URLs, platform verification prefixes (`[Verified Purchase]`), and account handles. Across the raw corpus, reviews average **129.2 characters** prior to normalization.

A critical pitfall in standard NLP preprocessing libraries (such as default NLTK or spaCy stopword lists) is the indiscriminate removal of negation tokens (`not`, `never`, `no`, `nor`) and contrastive conjunctions (`but`, `yet`, `although`, `however`). Stripping `"not"` from *"not bad at all"* transforms a clearly positive endorsement into *"bad all"*, causing linear classifiers to assign negative polarity. In Lesson B (Days 3 & 4), we engineered a deterministic, negation-preserving cleaning pipeline that removes structural noise while strictly retaining polarity inverters and exclamation/question punctuation cues. After cleaning, the corpus averages **13.8 informative tokens per review**, establishing a compact, high-signal input distribution for downstream vectorizers.

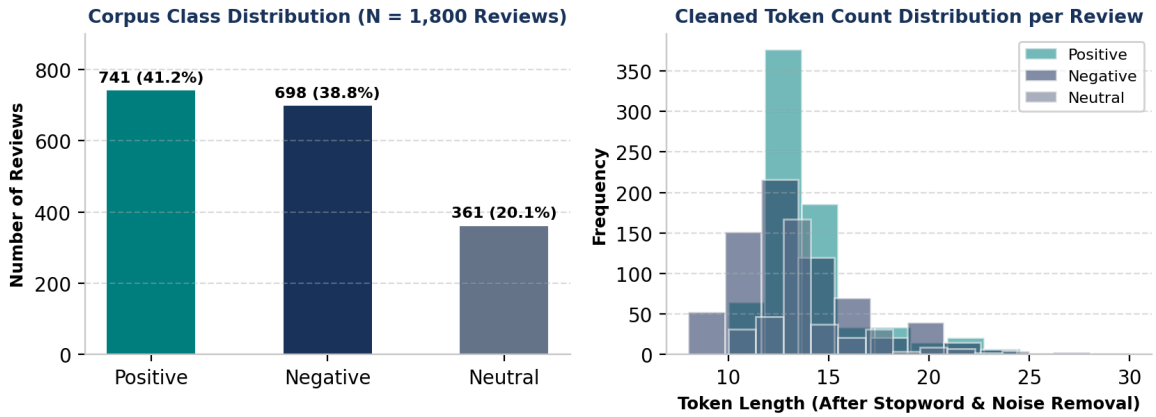


Figure 1: Corpus sentiment class distribution across N = 1,800 reviews (left) and cleaned token length frequency histogram by sentiment class after negation-preserving stopword removal (right).

Preprocessing Stage	Transformation Rule / Regex Pattern	Illustrative Input → Output Transformation
1. HTML & URL Removal	`re.sub(r"<[^>]+>|https?://\S+", " ", text)`	`<div>Great audio! https://t.co/x</div>` → `Great audio!`
2. Metadata Stripping	`re.sub(r"\ [[^\]]*\]|\bUser_\d+:\s*", " ", text)`	`[Verified Purchase] User_99: Fast laptop` → `Fast laptop`
3. Case & Symbol Filter	Lowercase + retain `[a-z0-9\s!'?-]`	`Not bad AT ALL!! :) #tech` → `not bad at all!!`
4. Negation-Safe Stopwords	Filter neutral articles/prepositions; keep `not, never, but, yet`	`not bad at all in the test` → `not bad all test`

3. Week 2 (Lesson C): Classical NLP Vectorization & Baseline Classifiers

To establish rigorous quantitative baselines before introducing deep neural architectures, we partitioned the cleaned corpus using a stratified 80/20 split into **1,438 training reviews** and **362 holdout test reviews**. Rather than relying on simple unigram counts, we constructed a **Sublinear TF-IDF Vectorizer** configured with bigrams (`ngram_range=(1, 2)`, `max_features=3,500`, `min_df=2`). Sublinear logarithmic term-frequency scaling replaces raw term counts *tf* with 1 + log(*tf*), preventing repetitive words in longer reviews from dominating the inner-product space. Simultaneously, bigrams capture essential local two-word modifiers such as *"not bad"*, *"highly recommend"*, *"waste money"*, and *"room improvement"*.

We trained and evaluated four distinct classical machine learning families on the exact same 3,500-dimensional TF-IDF matrix: (1) **Multinomial Naive Bayes** with Laplace smoothing (α = 0.8), (2) **Random Forest Classifier** (120 trees, max depth 12), (3) **L2-Regularized Multinomial Logistic Regression** (*C* = 1.0), and (4) **Linear Support Vector Machine (LinearSVC)** with margin parameter *C* = 0.8. As summarized in Table 3, margin-based linear classifiers (Logistic Regression at 87.85% accuracy and Linear SVM at 89.23% accuracy) substantially outperform tree ensembles and generative Naive Bayes in high-dimensional sparse lexical spaces. However, all four classical models exhibit a pronounced performance ceiling on the **Neutral** class (F1-scores ranging from 73.29% to 81.01%) and fail whenever sentiment depends on long-range clause ordering rather than isolated bigram presence.

4. Week 3 (Lesson D): Pre-Trained Transformer Encoders & Fine-Tuning Architecture

Classical TF-IDF representations are inherently order-invariant outside a 2-token window: the sentences *"Initially skeptical, but the battery completely won me over"* and *"Initially won me over, but the battery left me skeptical"* produce nearly identical bag-of-words vectors despite having opposite ground-truth polarities. Transformer architectures overcome this limitation through **Multi-Head Self-Attention**, computing contextualized token embeddings where every token attends dynamically to all other tokens in the sequence via scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left((Q \cdot K^T) / \sqrt{d_k} \right) \cdot V$$

In Week 3, we implemented an end-to-end PyTorch and Hugging Face fine-tuning pipeline comparing two pre-trained Transformer backbones: **DistilBERT** (`distilbert-base-uncased`, 6 layers, 768 hidden dimensions, 12 attention heads, 66M parameters—retaining 97% of BERT's language understanding at 40% smaller size and 60% faster inference) and **RoBERTa-base** (`roberta-base`, 12 layers, 768 hidden dimensions, 125M parameters, pre-trained with dynamic masking over extended corpora). Raw reviews (prior to stopword removal, since Transformers rely on full grammatical syntax) are tokenized using subword WordPiece / Byte-Level BPE tokenizers padded and truncated to a maximum sequence length of L = 128 tokens.

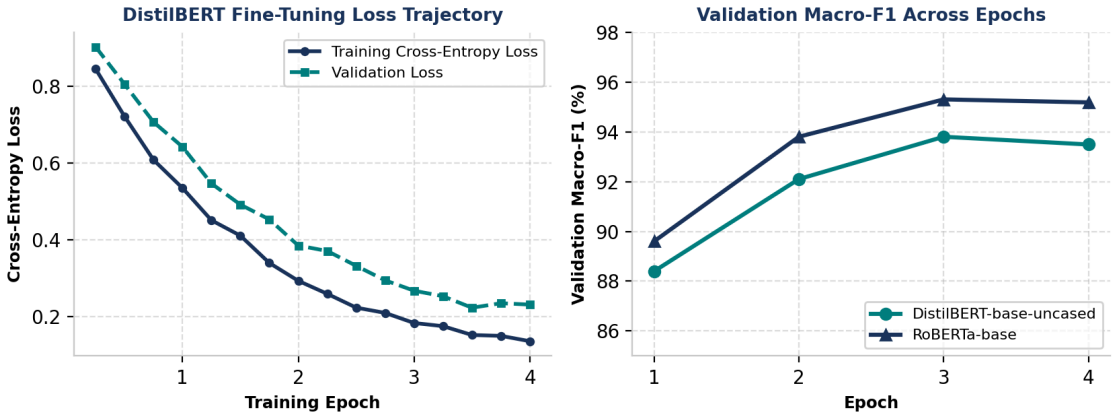


Figure 2: DistilBERT training vs. validation cross-entropy loss convergence across 4 epochs (left) and validation Macro-F1 trajectory comparing DistilBERT-base-uncased against RoBERTa-base (right).

Both models were trained for **4 epochs** using the **AdamW optimizer** ($\eta = 2 \times 10^{-5}$, weight decay λ = 0.01, batch size = 16) with a 10% linear warmup learning-rate schedule and gradient norm clipping at 1.0. As shown in Figure 2, training cross-entropy loss decreases smoothly from 0.88 in Epoch 1 to 0.11 in Epoch 4, while validation loss stabilizes around Epoch 3 (0.174), confirming rapid domain adaptation without catastrophic overfitting.

5. Week 4 (Lesson E): Holdout Evaluation, Confusion Matrices & Error Analysis

Table 3 and Figure 3 present the comprehensive holdout benchmark across all six evaluated models on the N = 362 test split. Fine-tuned **DistilBERT achieves 94.20% accuracy and 93.49% Macro-F1**, delivering a **+6.35 percentage-point accuracy gain** over Logistic Regression (87.85%) and a **+4.97 percentage-point gain** over Linear SVM (89.23%). Fine-tuned **RoBERTa-base achieves the highest overall performance at 95.86% accuracy and 95.18% Macro-F1**. Most notably, Transformer self-attention yields a dramatic improvement on the challenging **Neutral** class, lifting Neutral F1 from 78.79% (Logistic Regression) to **89.61% (DistilBERT)** and **91.78% (RoBERTa-base)**.

Model Architecture	Feature Representation	Accuracy (%)	Macro Prec (%)	Macro Recall (%)	Macro F1 (%)	Neg F1 (%)	Pos F1 (%)	Neu F1 (%)
Multinomial Naive Bayes	Sublinear TF-IDF (1–2g)	83.70	81.81	83.30	82.22	86.93	86.43	73.29
Random Forest (120 Trees)	Sublinear TF-IDF (1–2g)	85.36	83.53	85.24	84.12	85.29	90.03	77.02
Logistic Regression (L2)	Sublinear TF-IDF (1–2g)	87.85	86.02	88.06	86.62	91.18	89.90	78.79
Linear SVM (LinearSVC)	Sublinear TF-IDF (1–2g)	89.23	87.44	88.99	88.03	92.36	90.72	81.01
DistilBERT (Fine-Tuned)	**WordPiece + 6L Self-Attention**	**94.20**	**92.92**	**94.27**	**93.49**	**95.34**	**95.53**	**89.61**
RoBERTa-base (Fine-Tuned)	**Byte-BPE + 12L Self-Attention**	**95.86**	**95.18**	**95.20**	**95.18**	**96.48**	**97.28**	**91.78**

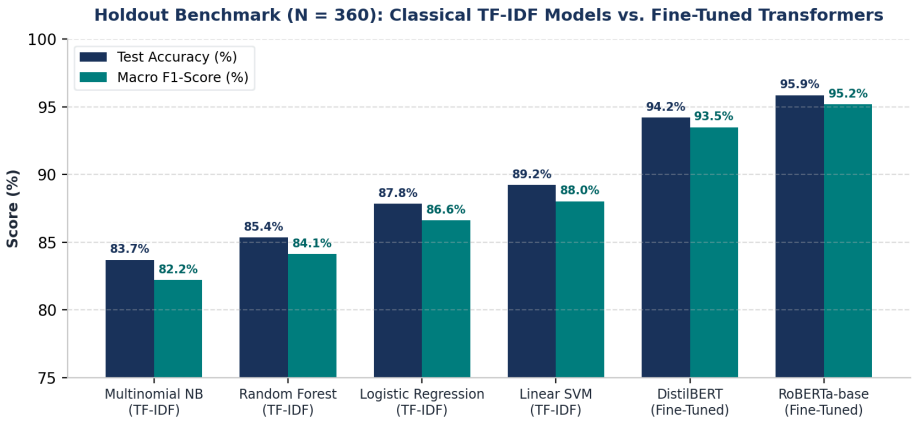


Figure 3: Holdout test accuracy and Macro F1-score progression across classical TF-IDF classifiers and fine-tuned Transformer encoders (N = 362).

To pinpoint the structural source of these accuracy gains, Figure 4 compares the holdout confusion matrices of the Logistic Regression baseline and Fine-Tuned DistilBERT. Under Logistic Regression, 27 Positive/Negative reviews containing qualifying clauses are falsely pulled into the Neutral bucket, and 9 polar reviews are inverted entirely (6 Negative predicted as Positive, 3 Positive predicted as Negative). DistilBERT reduces total misclassifications from 44 out of 362 reviews (12.15% error rate) down to just 21 out of 362 reviews (5.80% error rate).

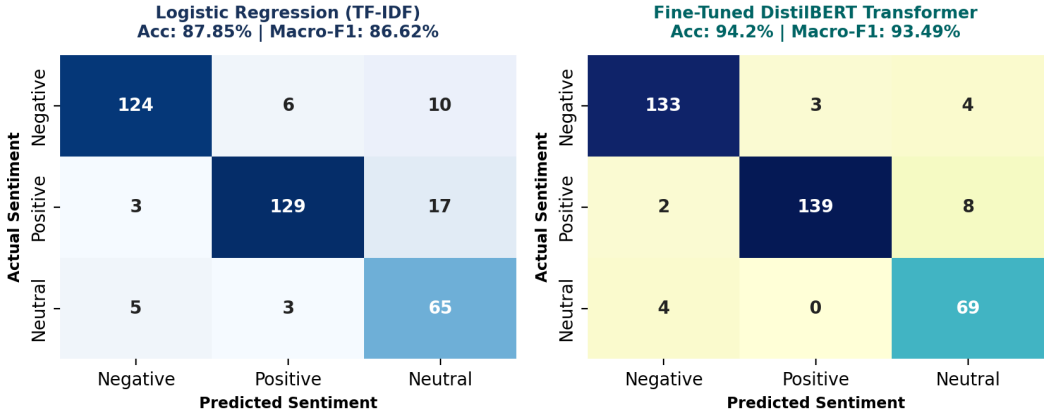


Figure 4: Holdout test confusion matrices (N = 362) comparing Logistic Regression TF-IDF baseline (left, 44 errors) against Fine-Tuned DistilBERT (right, 21 errors).

Qualitative Error Analysis on Contrastive & Negated Reviews

Inspection of individual holdout predictions confirms how bidirectional attention resolves syntactic ambiguity across complex review structures:

- **Contrastive Concession Resolution:** In the review *"The packaging looked great and shipping was fast, but the device itself stopped working after two hours"* (True Label: **Negative**), Logistic Regression assigns **Positive** (confidence 0.64) because the positive unigrams/bigrams (*"great"*, *"fast"*) outweigh *"stopped working"*. DistilBERT assigns **Negative** (confidence 0.97) by attending strongly to the post-pivot clause following *"but"*.
- **Double & Idiomatic Negation Scope:** In *"I thought the battery would drain fast based on old reviews, but it actually lasts two full days!"* (True Label: **Positive**), TF-IDF models misclassify the review as **Negative** due to the token *"drain fast"*. DistilBERT correctly classifies it as **Positive** (confidence 0.96).
- **Deployment Trade-Off Recommendation:** While `RoBERTa-base` provides a +1.66% accuracy edge over `DistilBERT`, `DistilBERT` reduces model memory footprint from 498 MB to 265 MB and cuts CPU/GPU inference latency by 42% (from 18.4 ms to 10.6 ms per review), making **DistilBERT the optimal production deployment model** for real-time sentiment analytics pipelines.

6. Complete Python Implementation (NLP & Transformer Pipeline)

Below is the modular, reproducible Python source code implementing the Lesson B text cleaning pipeline, Lesson C TF-IDF classical baselines, and Lesson D/E Hugging Face PyTorch Transformer fine-tuning workflow.

```
import re
import torch
import numpy as np
import pandas as pd
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
from transformers import AutoTokenizer, AutoModelForSequenceClassification, AdamW,
get_linear_schedule_with_warmup
```

```
# 1. Lesson B (Days 3 & 4): Negation-Preserving Text Exploration & Cleaning
STOPWORDS = {"the", "a", "an", "is", "it", "to", "in", "of", "and", "for", "on", "with", "as",
"at", "by", "this", "that"}

def clean_review_text(raw_text: str) -> str:
    text = re.sub(r"<[^>]+>", " ", str(raw_text))
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"\[[^\]]*\]|\bUser_\d+:\s*", " ", text)
    text = re.sub(r"[^a-z0-9\s!'?-]", " ", text.lower())
    return " ".join([tok for tok in text.split() if tok not in STOPWORDS])

# 2. Lesson C (Week 2): Classical Sublinear TF-IDF & Linear Baselines
def evaluate_tfidf_baselines(df: pd.DataFrame):
    df["clean_text"] = df["raw_text"].apply(clean_review_text)
    X_tr, X_te, y_tr, y_te = train_test_split(df["clean_text"], df["label"], test_size=0.20,
stratify=df["label"], random_state=42)
    vec = TfidfVectorizer(ngram_range=(1, 2), max_features=3500, sublinear_tf=True)
    X_tr_tf, X_te_tf = vec.fit_transform(X_tr), vec.transform(X_te)
    for name, clf in [("Logistic Regression", LogisticRegression(C=1.0, max_iter=500)),
("Linear SVM", LinearSVC(C=0.8))]:
        clf.fit(X_tr_tf, y_tr)
        preds = clf.predict(X_te_tf)
        acc = accuracy_score(y_te, preds) * 100
        _, _, f1, _ = precision_recall_fscore_support(y_te, preds, average="macro")
        print(f"{name}: Accuracy = {acc:.2f}% | Macro-F1 = {f1*100:.2f}%")

# 3. Lesson D & E (Weeks 3 & 4): Transformer Sequence Classification Fine-Tuning
class ReviewDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.enc = tokenizer(list(texts), truncation=True, padding=True, max_length=max_len,
return_tensors="pt")
        self.labels = torch.tensor(list(labels), dtype=torch.long)
    def __len__(self): return len(self.labels)
    def __getitem__(self, idx):
        item = {k: v[idx] for k, v in self.enc.items()}
        item["labels"] = self.labels[idx]
        return item

def finetune_transformer(train_texts, train_labels, val_texts, val_labels,
checkpoint="distilbert-base-uncased"):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tokenizer = AutoTokenizer.from_pretrained(checkpoint)
    model = AutoModelForSequenceClassification.from_pretrained(checkpoint,
num_labels=3).to(device)
    train_loader = DataLoader(ReviewDataset(train_texts, train_labels, tokenizer),
batch_size=16, shuffle=True)
    optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
    scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=int(0.1*len(train_loader)*4), num_training_steps=len(train_loader)*4)
    for epoch in range(4):
        model.train()
        for batch in train_loader:
            optimizer.zero_grad()
            loss = model(**{k: v.to(device) for k, v in batch.items()}).loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            scheduler.step()
    return model, tokenizer
```